
結果の取得と出力 その2

演習パッケージ: P10_Scoring2

Geant4 11.3.0準拠

Geant4講習会資料: HEP/NP/Space 2025



大学共同利用機関法人
高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

演習の目的

■ P10_Scoring2/Pixel_Four_Output

1. 磁場の下でのミュオンのトラッキングをして、その運動量を推測する。
 1. P06/Pixel_Four のジオメトリに磁場を加える
 2. ミュオンのトラッキングを行い、4つのPixel検出器への入射位置を求める。
 3. 要素の3つのレプリカ番号のtuple `replicaNo depth = 2, 1, 0`
 4. 世界座標のtuple `(x, y, z)`
2. イベント番号を取得して、上の二つと合わせてHit情報とする
3. Hitを集めてRecordとする
4. Record は独立したデータストア・クラスに蓄積する。
5. ランの終わりにRecordを取り出してフラットなCSVファイルに書き出す

■ CSV ファイルからヒストグラムや散布図を得る

1. jupyterでP10_Radius.ipynbを実行する

演習目次

[目次]

P10_Scoring2/Pixel_Four_Output

1. 目標の詳細検討と実行

1. マクロを実行してCSVファイルを出力

2. プログラムのキーポイント

- とにかくコードを読むのが主な課題である。

1. Geometry.cc のConstructSDandField() で磁場の入れ方を読む

2. SensitiveVolume::ProcessHits() をよんで、

- 荷電一次粒子の選択、入射点の在るPixel要素のレプリカ番号と世界座標を取得する

3. SimData.hh を読む

4. EndOfRunAction でRecord を読み出しCSVファイルに書き出す。

3. CSVファイルの解析

1. jupyterを使って解析する。

- 各ピクセルへの入射点の分布
- 半径の最小二乗法フィット

目標の具体化

設計概略

- Pixel_Fourを少し変更したジオメトリーを用いて運動量測定を試みる。
 - ジオメトリー
 - ・ 4枚のシリコン検出器を同じ方向に揃える。
 - ・ Y方向に一様な磁場をかける
 - 有感検出器
 - ・ 一つのセルの論理ボリュームをSensitiveVolumeとする。
 - ・ シリコンのコピー番号(1001~1004), シリコンのなかでの2つのコピー番号を取得する。
 - ・ 各シリコンへの粒子の入射位置の世界座標を求める。
 - ・ 粒子が多重散乱以外の相互作用で4枚すべてのシリコンを通過しない場合にも対応できるようにする
 - ・ イベントはそのイベント番号で識別する。
 - データは独立したデータストア SimData に格納する
 - ・ eventID, tuple of Replica Copy No, tuple of world (x, y, z) のvector
 - ・ EndofRun でCSVファイルに出力する
 - データ解析にはjupyter lab を使う。
 - ・ 最小二乗法で円にフィットし、その半径と誤差をもとめる。
 - ・ 多重散乱が分かりやすいいくつかのプロットを作成する

P10_Scoring2/Pixel_Four_Output プログラムのビルド

課題: まずは動かしてみよう. 手順は同じ。

P10_Scoring2/Pixel_Four_Outputのソースコードをビルドして実行する

1) P10_Scoring2/Pixel_Four_Outputのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2024/P10_Scoring2/Pixel_Four_Output
$ ls
source/  util/  TestBench/
```

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

Scriptを走らせるとアプリ実行環境が完成

ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P10_Scoring2/Pixel_Four_Outputプログラムを実行する

課題:1 実行されたプログラムを使う マクロを実行する

1) Qt画面のコマンド窓に以下を入力しアプリ実行

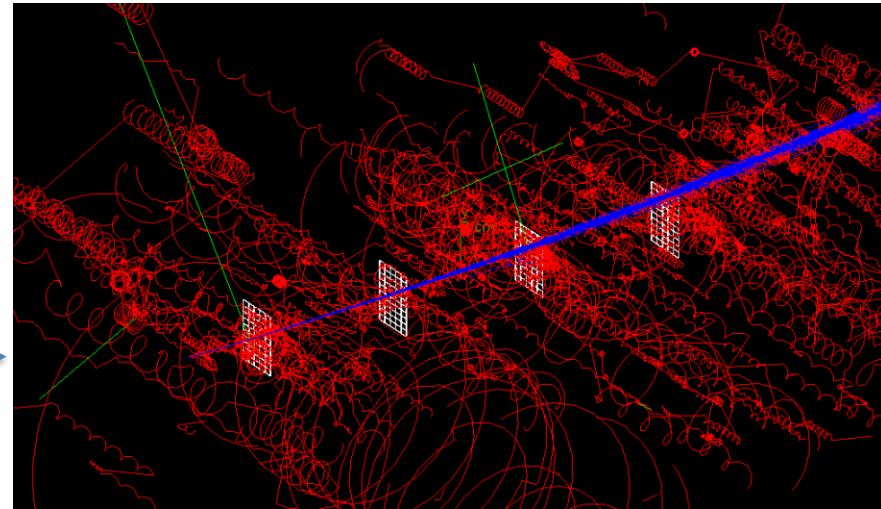
```
/control/execute muon.mac
```

500 MeV μ^+ 1イベントだけ

2) Qtコマンド窓に以下のコマンドを入力

```
/run/beamOn 2000
```

端末出力には、eventide, replicaNos, world xyz



2) アプリを終了

```
exit
```

```
% ls
```

```
P10_Pixe_Four_Output.csv
```

後で解析するCSVファイルを確認する

```
=== ProcessHits ===
eventID = 1999 Track ID =: 1 Particle =: mu+ time = 0.085030698878892
### entry point ###
replicaNo = 2 replicaNo_1 = 5 replicaNo_2 = 1000 x = 0 , y = 0.15586494971672 , z = -75.1
=== ProcessHits ===
eventID = 1999 Track ID =: 1 Particle =: mu+ time = 0.25447281357319
### entry point ###
replicaNo = 2 replicaNo_1 = 5 replicaNo_2 = 1001 x = -0.023575097602258 , y = 1.4576465467851 , z = -25.1
=== ProcessHits ===
eventID = 1999 Track ID =: 1 Particle =: mu+ time = 0.42408722251864
### entry point ###
replicaNo = 2 replicaNo_1 = 7 replicaNo_2 = 1002 x = -0.027601432877086 , y = 4.0559418858454 , z = 24.9
=== ProcessHits ===
eventID = 1999 Track ID =: 1 Particle =: mu+ time = 0.5939665528066
### entry point ###
replicaNo = 2 replicaNo_1 = 8 replicaNo_2 = 1003 x = 0.053102117927959 , y = 7.8698125483737 , z = 74.9
Run terminated.
Run Summary
Number of events processed : 2000
User=8.310000s Real=8.623778s Sys=1.030000s [Cpu=108.3%]
```

次の課題で、CSVファイルの解析を行う

課題: 端末出力を読む muon.mac: μ^+ 500 MeV 1 event

Run 1 starts.

=== ProcessHits ===

eventID = 0 Track ID =: 1 Particle =: mu+ time = 0.085030731110496

entry point

replicaNo = 2 replicaNo_1 = 5 replicaNo_2 = 1000 x = 0 , y = 0.15586494510242 , z = -75.1

=== ProcessHits ===

eventID = 0 Track ID =: 1 Particle =: mu+ time = 0.25446976153106

entry point

replicaNo = 2 replicaNo_1 = 5 replicaNo_2 = 1001 x = 0.14865009727351 , y = 1.4134949622788 , z = -25.1

=== ProcessHits ===

eventID = 0 Track ID =: 1 Particle =: mu+ time = 0.42406757814163

entry point

replicaNo = 2 replicaNo_1 = 6 replicaNo_2 = 1002 x = 0.25197893168313 , y = 3.912868780485 , z = 24.9

=== ProcessHits ===

eventID = 0 Track ID =: 1 Particle =: mu+ time = 0.5939402902565

entry point

replicaNo = 2 replicaNo_1 = 8 replicaNo_2 = 1003 x = 0.36035994302691 , y = 7.6992489516235 , z = 74.9

Run terminated.

Run Summary

Number of events processed : 1

入射点のZ座標はPixelの1mmの厚さだけ移動距離とずれる

Z方向に { -7.5*cm, -2.5*cm, +2.5*cm, +7.5*cm } 移動

Y方向に20mm/10分割 => replicaNo_1
X方向に15mm/5分割 => replicaNo

CSVファイルの解析

出力されたファイル P10_Pixel_Four_Output.csv

%more P10_Pixel_Four_Output.csv 各列の内容を確認する

イベント #0	{	0	2	5	1000	0	0.155865	-75.1
		0	2	5	1001	-0.02021	1.43914	-25.1
		0	2	6	1002	-0.09219	3.9299	24.9
		0	2	8	1003	-0.13966	7.70737	74.9
イベント #1	{	1	2	5	1000	0	0.155854	-75.1
		1	2	5	1001	-0.02233	1.39609	-25.1
		1	2	6	1002	-0.02424	3.94263	24.9
		1	2	8	1003	-0.01971	7.71104	74.9
イベント #2	{	2	2	5	1000	0	0.155865	-75.1
		2	2	5	1001	0.02762	1.41808	-25.1
		2	2	6	1002	0.073383	3.95465	24.9
		2	2	8	1003	0.130628	7.74077	74.9
イベント #3	{	3	2	5	1000	0	0.155865	-75.1
		3	2	5	1001	-0.00439	1.39705	-25.1
		3	2	6	1002	0.045601	3.86593	24.9
		3	2	8	1003	0.102143	7.586	74.9

Event No

レプリカ番号

世界座標

X

Y

Z

世界座標値のヒストグラムとxy散布図を得たい

円のLSQ fitに使うデータは(Z, Y)

課題: Jupyter lab を起動しスクリプトを実行する

- ✓ P10_Scoring2/Pixel_Four_Output/TestBench の下にCSVファイルとスクリプトファイル P10_Radius.ipynbがあることを確認する。
- 1. **TestBenchの中で**次のコマンドをタイプすると、ブラウザに新しいタブが開く
 - \$ **jupyter lab** 実行開始
 - [I 21:11:58.858 LabApp] JupyterLab extension loaded from /home/g4user/.pyenv/versions/anaconda3/lib/python3.7/site-packages/jupyterlab
 - [I 21:11:58.861 LabApp] JupyterLab application directory is /home/g4user/.pyenv/version/anaconda3/share/jupyter/lab
 - [I 21:11:58.874 LabApp] Serving notebooks from local directory:
- 2. 少し待つとブラウザに新しいタブが開き、すべての操作はブラウザ上で行う
- 3. 左側のペインにJupyter lab を起動したディレクトリにあるファイル一覧が表示される。
- 4. CSVファイルをクリックして読み込むと新しいサブタブが開き、内容が表示される
- 5. **P10Radius.ipynb**をクリックしてサブタブを開く

← URL

課題: P10_Radius.ipynb に目を通す

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
```

お決まりの道具

```
def calculate_stats(data, column):
    mean_value = data[column].mean()
    std_value = data[column].std()
    return mean_value, std_value
```

```
def plot_histogram(data, column, bins, xlabel, ylabel, title, filename):
    plt.hist(data[column], bins=bins)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(title)
    plt.savefig(filename)
    plt.show()
```

統計計算

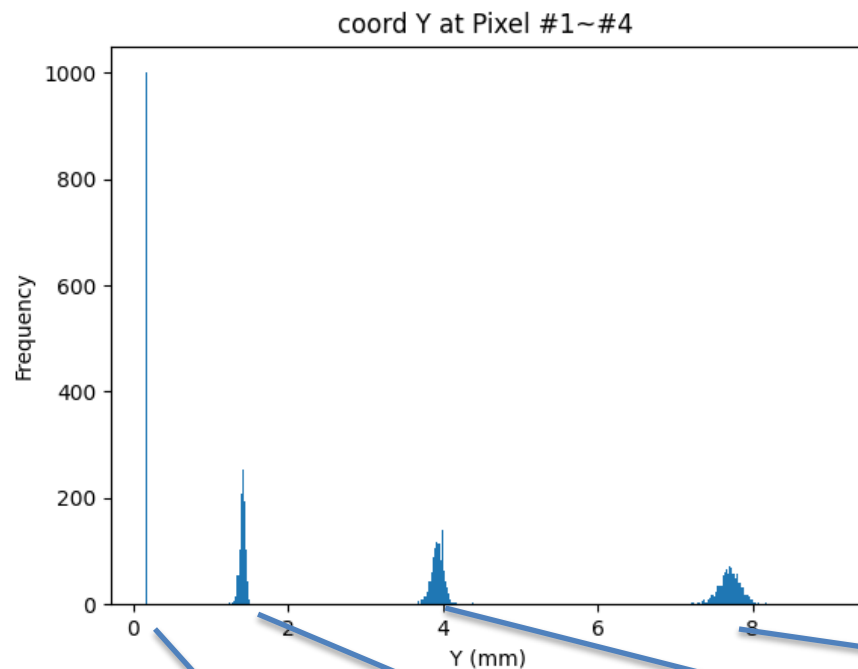
ヒストグラム作成

散布図作成

ルーチン

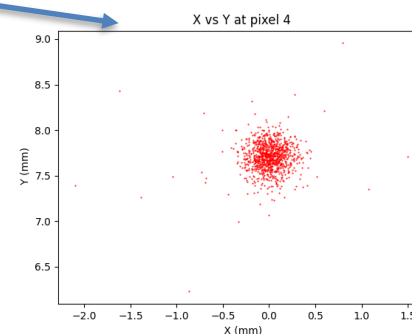
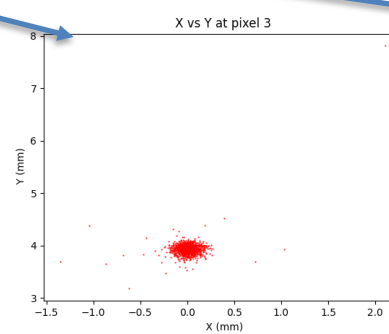
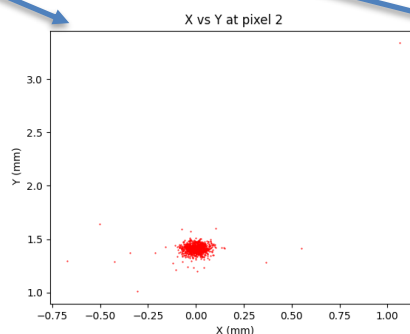
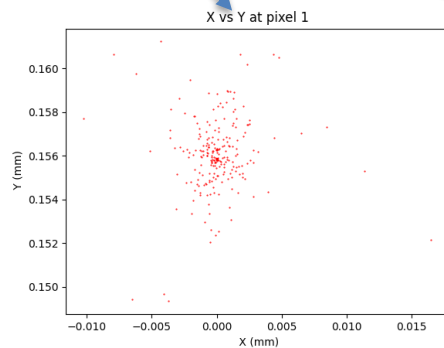
```
def plot_scatter(data, x_column, y_column, xlabel, ylabel, title, filename):
    plt.scatter(data[x_column], data[y_column], c="red", s=1, marker=".")
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(title)
    plt.savefig(filename)
    plt.show()
```

課題: Runして多重散乱の様子を見る



Y at Pixel 1000, mean = 0.1559, sigma = 0.0011
X at Pixel 1000, mean = 0.0000, sigma = 0.0017
Y at Pixel 1001, mean = 1.4106, sigma = 0.0491
X at Pixel 1001, mean = 0.0017, sigma = 0.0479
Y at Pixel 1002, mean = 3.9281, sigma = 0.1239
X at Pixel 1002, mean = 0.0036, sigma = 0.1071
Y at Pixel 1003, mean = 7.7050, sigma = 0.1968
X at Pixel 1003, mean = 0.0054, sigma = 0.1785
Circle Center = (1990.4315, -100.0855),
Radius = 1990.4328, Radius Error = 0.1186

半径の計算値は次のページに



検討

- ここまでの解析、つまり
 - 世界座標の1次元ヒストグラム
 - 世界座標の2次元プロット
 - 各ピクセルごとの世界座標の平均値と σ の計算
 - ✓ などはランの前にG4AnalysisManager で定義しておき、イベントごとにヒストやプロットにFillするのと同じことである。
- この演習プログラムでは、すべての生スコアをCSVファイルに出力してからPython matplotlib を使って解析し表示した。(G4AnalysisManagerでもNtuple を使って同じような方法が使える。)
 - この手法では、個別のヒストやプロットからは得られない解析、つまりヒストグラムに表示されているデータの元の生スコアを使って最小二乗法フィットなどができる。

半径の計算 Geminiに計算させた

500MeV μ^- B=1T での半径の計算はChatGPT, Geminiは正しく同じ答えを出した。
Geminiの回答は以下の通り。

ミュオンのトラジェクトリーの円の半径を計算するには、以下の手順が必要です。

1. 相対論的運動量の計算

- * ミュオンの静止質量: $m \approx 105.7 \text{ MeV}/c^2$
- * ミュオンの全エネルギー: $E = \text{運動エネルギー} + \text{静止エネルギー} = 500 \text{ MeV} + 105.7 \text{ MeV} = 605.7 \text{ MeV}$
- * 相対論的運動量: $p = \sqrt{E^2 - (mc^2)^2} / c = \sqrt{(605.7^2 - 105.7^2)} / c \approx 596.5 \text{ MeV}/c$.

2. 円運動の半径の計算

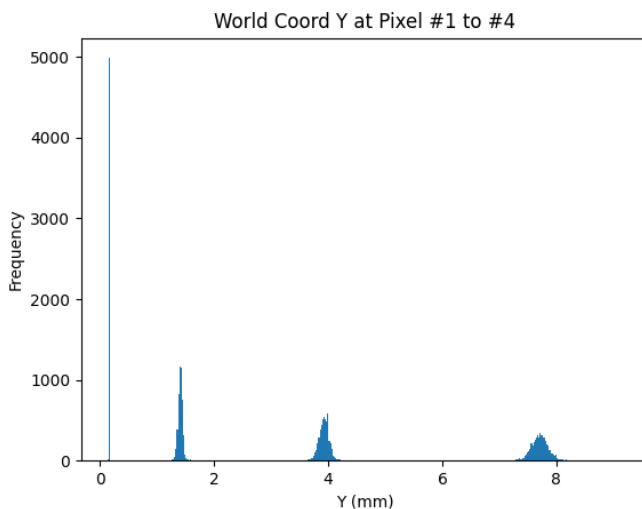
- 磁場: $B = 1 \text{ T}$
- ミュオンの電荷: $q = e$ (素電荷)
- * 円運動の半径: $r = p / (qB)$ 単位を変換して計算します。
- * $p \approx 596.5 \text{ MeV}/c = 596.5 \times 10^6 \text{ eV} / c$
- * $1 \text{ eV} = 1.602 \times 10^{-19} \text{ J}$
- * $c \approx 3 \times 10^8 \text{ m/s}$
- * $e \approx 1.602 \times 10^{-19} \text{ C}$
 $r = (596.5 \times 10^6 \times 1.602 \times 10^{-19} \text{ J} / (3 \times 10^8 \text{ m/s})) / (1.602 \times 10^{-19} \text{ C} \times 1 \text{ T}) \approx 1.99 \text{ m}$

3. したがって、運動エネルギー500MeVのミュオンが1Tの磁場を通るときのトラジェクトリーの円の半径は約1.99mです。

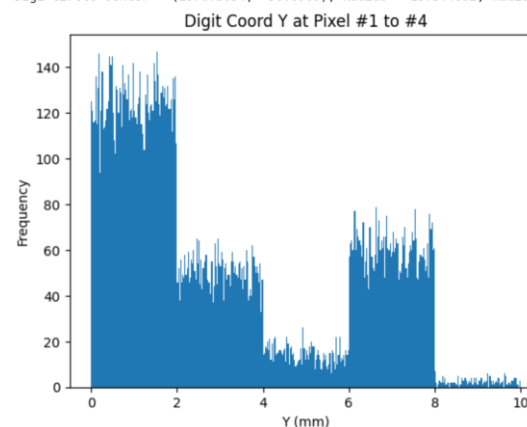
ピクセル測定器のコピー番号から半径を推定する

- ここまでは入射点の世界座標から回転半径を求めた。つまり、測定誤差はゼロとして、多重散乱による誤差だけを考慮した。
- Digitization: Y方向には2mm刻みで10にreplicationされている。世界座標でなく、レプリカの中心Y座標値をもとにしてピクセルの半幅1mmを一様分布してY座標値とする。
- 課題: P10_Randomized.ipynb をRunする

最小自乗近似で入射点の世界座標(下左図)を使うと1990mm だが、コピー番号から座標(下右図)を推定した場合には1375mmとなった。



Circle Center = (1990.4315, -100.0855), Radius = 1990.4328, Radius Error = 0.1186
Digi Circle Center = (1376.3054, -56.6908), Radius = 1375.4832, Radius Error = 0.7472



Visual Studio code でのjupyter lab 実行環境

The screenshot shows the Visual Studio Code interface with a Jupyter Notebook open. The Explorer sidebar on the left lists files in the 'TESTBENCH' directory. The main editor window shows the 'P10_Radius.ipynb' notebook with the following code:

```
import pandas as pd

# Plot histogram of coordinate Y including all pixels
plot_histogram(df, 5, bins=400, xlabel="Y (mm)", ylabel="Frequency", title="Coord Y at Pixel #1 to #4")
plot_histogram(df, 2, bins=50, xlabel="copyNo_Y", ylabel="Frequency", title="copyNo of Y")
# Plot scatter plots for each pixel replica
for pixel_id, data in pixel_replicas.items():
    plot_scatter(data, 4, 5, xlabel="X (mm)", ylabel="Y (mm)", title=f"X at Pixel {pixel_id}")
```

The output of the notebook shows statistical data for four pixels:

```
[1] ✓ 2.9s Python
...
Y at Pixel 1000, mean = 0.1559, sigma = 0.0013
X at Pixel 1000, mean = 0.0001, sigma = 0.0014
Y at Pixel 1001, mean = 1.4121, sigma = 0.0444
X at Pixel 1001, mean = 0.0019, sigma = 0.0482
Y at Pixel 1002, mean = 3.9304, sigma = 0.1033
X at Pixel 1002, mean = 0.0043, sigma = 0.1069
Y at Pixel 1003, mean = 7.7077, sigma = 0.1797
X at Pixel 1003, mean = 0.0058, sigma = 0.1770
Circle Center = (1991.3854, -100.1693), Radius = 1991.3877, Radius Error = 0
```

A histogram titled 'Coord Y at Pixel #1 to #4' is displayed, showing the frequency of Y coordinates. The x-axis is labeled 'Y (mm)' and ranges from 0 to 8. The y-axis is labeled 'Frequency' and ranges from 0 to 2000. The histogram shows four distinct peaks corresponding to the four pixels.

The Copilot sidebar on the right displays a 'Welcome to Copilot' message. At the bottom, a dialog box prompts for 'Signing in to github.com...' with a 'Cancel' button.

プログラムのキーポイントを理解する

課題: Geometry のポイントを読む

- SensitiveDetector と磁場を作るにはConstruct でなくてConstructSDandField

```
auto fieldValue = G4ThreeVector{ 0.0, 1.0*tesla, 0.0 };  
fMagFieldMessenger = new G4GlobalMagFieldMessenger(fieldValue);
```

- 独立したデータストアSimDataを使う。
 - SensitiveDetectorのインスタンスを作る前に知らせておくこと

```
SensitiveVolume* aSensiVol = new SensitiveVolume{"Pixel"};  
aSensiVol->SetSimData(simdata);
```

- Pixel Element のLogical Volume をSensitiveDetector にアサインする

```
SetSensitiveDetector("LogVol_PixElmt", aSensiVol, true );
```

課題: %more src/Geometry.cc

5cm置きに4枚
平行に置く

ConstructSDandField()
磁場と有感指定

Y方向の一様磁場
1 T

独立データストアの指定

LogVol_PixElmt
を有感指定

```
// Z positions and setting angles of four detectors
G4double z_pos[4] = { -7.5*cm, -2.5*cm, +2.5*cm, +7.5*cm };
G4double z_ang[4] = { 0.0*deg, 0.0*deg, 0.0*deg, 0.0*deg };
```

```
//-----
void Geometry::ConstructSDandField()
//-----
{
// Add uniform magnetic field to the world Y方向の一様磁場
auto fieldValue = G4ThreeVector{ 0.0, 1.0*tesla, 0.0 };
fMagFieldMessenger = new G4GlobalMagFieldMessenger(fieldValue);
fMagFieldMessenger->SetVerboseLevel(1);

// Register the field messenger for deleting
G4AutoDelete::Register(fMagFieldMessenger);

//Sensitive Volume 有感検出器

SensitiveVolume* aSensiVol = new SensitiveVolume{"Pixel"};
aSensiVol->SetSimData(simdata);
auto SDMan = G4SDManager::GetSDMpointer();
SDMan->AddNewDetector(aSensiVol);
SetSensitiveDetector("LogVol_PixElmt", aSensiVol, true );
```

課題: %more include/Geometry.hh

```
#include "G4VUserDetectorConstruction.hh"

class G4VPhysicalVolume;
class SimData;
class G4GlobalMagFieldMessenger;

//-----
class Geometry : public G4VUserDetectorConstruction
//-----
{
public:
    Geometry();
    ~Geometry() override;
    G4VPhysicalVolume* Construct() override;
    void ConstructSDandField() override;

    void SetSimData(SimData* data);
private:
    SimData* simdata ;
    static G4ThreadLocal G4GlobalMagFieldMessenger* fMagFieldMessenger;
};

inline void Geometry::SetSimData(SimData* data)
{
    G4cout << "Geometry::SetSimData " << G4endl;
    simdata = data ;
}
#endif
```

磁場と有感検出器

独立したデータストア

課題: SensitiveVolume のポイントを読む

- 一次粒子だけを選別する。

```
if (idTrack != 1) return false;          一次粒子を選ぶ  
if(track->GetTrackStatus()!=fAlive) return false;
```

- 念のためにイベント番号を記録する

```
auto runManager = G4RunManager::GetRunManager();  
auto currentEv = runManager->GetCurrentEvent();
```

- 入射ステップを選別する

```
if ( preStepPoint->GetStepStatus() != fGeomBoundary) return false;  一次粒子がピクセルに入射した
```

- 世界座標

```
position_World = preStepPoint->GetPosition();      世界座標ゲット
```

- コピー番号

```
replicaNo = theTouchable->GetReplicaNumber();  
G4int replicaNo_1 = theTouchable->GetReplicaNumber(1);  
replicaNo_2 = theTouchable->GetReplicaNumber(2);
```

SensitiveVolume ProcessHits()

課題: SensitiveVolume クラスを読む。

```
G4bool SensitiveVolume::ProcessHits(G4Step* aStep, G4TouchableHistory*)
{
  G4cout << "=== ProcessHits ===" << G4endl;
  G4double edep = aStep->GetTotalEnergyDeposit();
  if ( edep == 0. ) return false;

  // Retrieve information from the track object
  auto track = aStep->GetTrack();
  G4int idTrack = track->GetTrackID();
  // Primary track is picked up
  if (idTrack != 1) return false;          一次粒子を選ぶ
  nStep = track->GetCurrentStepNumber();
  G4String particleName = track->GetDefinition()->GetParticleName();
  // primary particle has just entered a pixel
  if(track->GetTrackStatus()!=fAlive) return false;

  G4double globalTime = track->GetGlobalTime();
  auto runManager = G4RunManager::GetRunManager();
  auto currentEv = runManager->GetCurrentEvent();
  G4int eventID = currentEv->GetEventID();    イベント番号を取得しておく

  G4cout << "  eventID = " << eventID << " Track ID = " << idTrack
    << " Particle = " << particleName << " time = " << globalTime <<G4endl;
```

Muon をトラッキングしたいので
一次粒子を選び、さらにその
トラックが死んでいない場合を選ぶ

課題: World coordinates and replica numbersの取得

```
// Get PreStepPoint and TouchableHandle objects
```

```
G4StepPoint* preStepPoint=aStep->GetPreStepPoint();
```

```
G4StepPoint* postStepPoint=aStep->GetPostStepPoint();
```

入射点の世界座標、
レプリカ番号を取得

```
// a primary track entry coordinates
```

```
if ( preStepPoint->GetStepStatus() != fGeomBoundary) return false; 一次粒子がピクセルに入射した  
ステップを選ぶ
```

```
G4ThreeVector position_World = preStepPoint->GetPosition();
```

世界座標ゲット

```
position_Worldx = position_World.x() ;
```

```
position_Worldy = position_World.y() ;
```

```
position_Worldz = position_World.z() ;
```

```
// Touchable information: Volume name and copy# at the current step
```

階層ごとのレプリカ番号ゲット

```
G4TouchableHandle theTouchable = preStepPoint->GetTouchableHandle();
```

```
G4String volumeName = theTouchable->GetVolume()->GetName();
```

```
G4int replicaNo = theTouchable->GetReplicaNumber();
```

```
G4String volumeName_1 = theTouchable->GetVolume(1)->GetName();
```

```
G4int replicaNo_1 = theTouchable->GetReplicaNumber(1);
```

```
G4String volumeName_2 = theTouchable->GetVolume(2)->GetName();
```

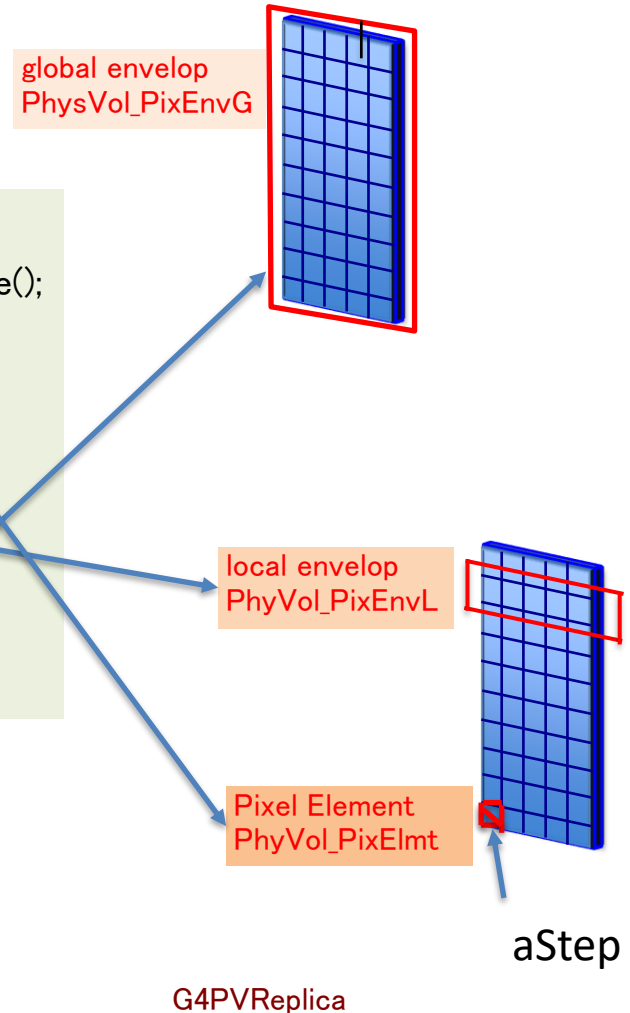
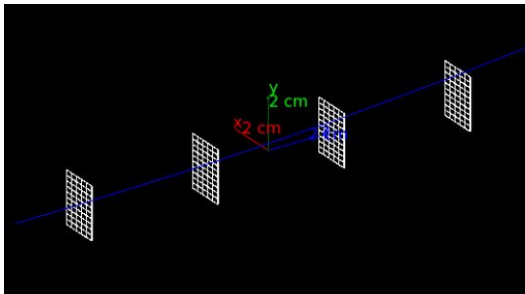
```
G4int replicaNo_2 = theTouchable->GetReplicaNumber(2);
```

階層構造のジオメトリ情報の取得 G4TouchableHandle

課題: Geometry.cc と SensitiveVolume.cc を見比べてタッチャブルの使い方を理解する

G4TouchableHandle を使って
階層構造のコピー番号を直接入手する例
depth = 0, 1, 2

```
G4StepPoint* preStepPoint=aStep->GetPreStepPoint();  
G4TouchableHandle theTouchable = preStepPoint->GetTouchableHandle();  
//pixel element  
G4String volumeName = theTouchable->GetVolume()->GetName();  
G4int reolcaNo = theTouchable->GetReplicaNumber();  
//local envelope  
EnvLVVolumeName = theTouchable->GetVolume(1)->GetName();  
    replicaNo_1No = theTouchable->GetReplicaNumber(1);  
// global envelope  
EnvGVVolumeName = theTouchable->GetVolume(2)->GetName();  
    replicaNo_2 = theTouchable->GetReplicaNumber(2);
```



課題: ProcessHits() 最後の部分Simdata.fill()を読む

```
// next lines for debug only

G4cout << "### entry point ####" <<G4endl;
G4cout << " replicaNo = " <<replicaNo
    << " replicaNo_1 = " << replicaNo_1
    << " replicaNo_2 = " << replicaNo_2
    <<" x = " << position_Worldx << " , y = " << position_Worldy
    << " , z = " << position_Worldz
    << G4endl;

// store thsi Hit   ヒット集合 をFill (int, {tuple}, {tuple})
simdata[0].Fill(eventID, {replicaNo, replicaNo_1, replicaNo_2}, {position_Worldx , position_Worldy,
position_Worldz});

return true;
}
```

イベントごとのヒット集合を
SimDataにFillする。データ構造は
SimData.hhにある

```
// SimData.hh Serial version
#ifndef SimData_h
#define SimData_h 1
#include "globals.hh"
#include <vector>
#include <tuple>
using Replica = std::tuple<G4int, G4int, G4int>;
using Coordinate = std::tuple<G4double, G4double, G4double>;
struct Hit {
    G4int eventID;
    Replica replica;
    Coordinate coordinate;
};

class SimData{
public :
    SimData() = default;
    ~SimData() = default;
    SimData( const SimData& ) = delete;
    void operator = (const SimData& ) = delete;
    void Fill (G4int eventID, Replica replica, Coordinate coordinate );
    void Initialize();
    const std::vector<Hit>& GetRecord();

private :
    std::vector<Hit> Record ;
    inline void SimData::Fill(G4int eventID, Replica replica, Coordinate coord)
    { Record.push_back({eventID, replica, coord}); }
    inline void SimData::Initialize()
    { Record.clear(); }
    inline const std::vector<Hit>& SimData::GetRecord() { return Record ; }

#endif
```

3つのレプリカ番号と3つの世界座標をそれぞれタプルにする

イベント番号、二つのタプルの集合を Hit とする

イベントごとのHitをRecordというvectorに追記する `push_back()`

タプルは要素の順序を気にしないので注意する

Fill()はProcessHits()中で使う
Initialize() はBeginOfRunAction()で
GetRecord()はEndOfRunActionで使う

課題: RunAction.cc 本体部分 を読む

```
void RunAction::EndOfRunAction(const G4Run* )
{
    G4String fName = "P10_Pixel_Four_Output.csv";
    auto record = simdata[0].GetRecord();

    std::ofstream outputfile(fName);
    for( auto val : record)
    {
        outputfile << (val.eventID);
        outputfile << " , " ;
        outputfile << std::get<0>(val.replica);
        outputfile << " , " ;
        outputfile << std::get<1>(val.replica);
        outputfile << " , " ;
        outputfile << std::get<2>(val.replica);
        outputfile << " , " ;
        outputfile << std::get<0>(val.coordinate);
        outputfile << " , " ;
        outputfile << std::get<1>(val.coordinate);
        outputfile << " , " ;
        outputfile << std::get<2>(val.coordinate);
        outputfile << std::endl ;
    }
    outputfile.close();
}
```

イベント番号

レプリカ番号のタプル

世界座標のタプル

課題: Application_Main.cc 主要部 順序に注意して読む

```
// Construct the "Serial" run manager
auto* runManager = G4RunManagerFactory::CreateRunManager(G4RunManagerType::Serial);

// Set up mandatory user initialization: Geometry
//   Set SimData pointer
//   use array of size 1
//   G4int nvec = 1 ;
auto simdata = new SimData[nvec];
auto geometry = new Geometry{};
geometry->SetSimData(simdata);
runManager->SetUserInitialization( geometry );

// Set up mandatory user initialization: Physics-List
runManager->SetUserInitialization( new FTFP_BERT{} );

// Set up user initialization: User Actions

auto useraction = new UserActionInitialization{};
useraction -> SetSimData(simdata) ;
runManager->SetUserInitialization( useraction );

// Initialize G4 kernel
runManager->Initialize();
```

演習 終了

付録： LLMの効用

- この演習で使ったPython scriptは手作りのものを丸々ChatGPTに渡し、改良を依頼した結果である。詳しいことは何も言わず、改良するように言っただけで、モジュール化を実行した。
- さらに、世界座標の最小二乗法フィットを加えるようにすること、コピー番号をもとに一樣乱数を振って世界座標値に代えて最小二乗法をすることもやらせた。
- ミューオンのトラジェクトリの半径の計算もやらせた。
- ここで紹介したのはうまくいったものだけであるが、指示をよく理解できないで何回も間違ったこともあった。その都度こちらの意図をより詳しく展開して再指示することで答えに行き着いた。例えば、トラジェクトリの半径の計算のような単純な問題でも、LLMによっては繰り返し間違えることがあった。
- Geant4のコード生成でもLLMで共通して間違ったのは実行の順序である。一応各クラスの順序はしておいたが、よく理解していないことがあった。
- にもかかわらず、クラス間の関係についてはよく理解してC++の文法を理解してクラス定義ファイルと実装ファイルを提出した。
- 大事なことは現時点でもLLMはデバッガーやコード解析ツール同様、ツールであって、その正誤も著作権も使い手の責任であることだ。将来はどうかかわらないが。今は、LLM自身がそういつている。

続き

```
def fit_circle(x, y):  
    def calc_radius(c):  
        return np.sqrt((x - c[0])**2 + (y - c[1])**2)  
  
    def fun(c):  
        ri = calc_radius(c)  
        return ri - ri.mean()  
  
    center_estimate = np.mean(x), np.mean(y)  
    result = least_squares(fun, center_estimate)  
    center = result.x  
    radius = calc_radius(center).mean()  
    residuals = calc_radius(center) - radius  
    radius_error = np.sqrt(np.sum(residuals**2) / len(residuals))  
    return center, radius, radius_error
```

最小自乗法フィット

```
# Read the CSV file  
df = pd.read_csv("P10_Pixel_Four_Output.csv", header=None)  
  
# Extract data frames according to the Pixel replica number in column 3  
pixel_replicas = {  
    1000: df[df.iloc[:, 3] == 1000],  
    1001: df[df.iloc[:, 3] == 1001],  
    1002: df[df.iloc[:, 3] == 1002],  
    1003: df[df.iloc[:, 3] == 1003]  
}
```

CSVファイルを読み込みdfへ

4枚のシリコンで」のデータを分離

最後

```
# Calculate and print statistics for each pixel replica
for pixel_id, data in pixel_replicas.items():
    mean_y, std_y = calculate_stats(data, 5)
    print(f"Y at Pixel {pixel_id}, mean = {mean_y:.4f}, sigma = {std_y:.4f}")

    mean_x, std_x = calculate_stats(data, 4)
    print(f"X at Pixel {pixel_id}, mean = {mean_x:.4f}, sigma = {std_x:.4f}")

# Perform least squares fit for all data points in df[5] and df[6]
center, radius, radius_error = fit_circle(df[5], df[6])
print(f"Circle Center = ({center[0]:.4f}, {center[1]:.4f}), ¥
      Radius = {radius:.4f}, Radius Error = {radius_error:.4f}")

# Plot histogram of coordinate Y including all pixels
plot_histogram(df, 5, bins=400, xlabel="Y (mm)", ylabel="Frequency", ¥
              title="Coord Y at Pixel #1 to #4", filename="WorldY")

# Plot scatter plots for each pixel replica
for pixel_id, data in pixel_replicas.items():
    plot_scatter(data, 4, 5, xlabel="X (mm)", ylabel="Y (mm)", ¥
                title=f"X vs Y at Pixel {pixel_id}", filename=f"scatterXY_{pixel_id}")
```

Y座標値

X座標値

すべての(y, z)から
円弧フィット

最小二乗法フィット

- 与えられた点 (x, y) の集合に対して最小二乗法を用いて円をフィットして、その中心座標 $(c[0], c[1])$ 、半径、および半径の誤差を計算する関数

```
import numpy as np
from scipy.optimize import least_squares

def fit_circle(x, y):
    # 円の中心  $(c[0], c[1])$  と各点  $(x, y)$  から円の半径を求める
    def calc_radius(c):
        return np.sqrt((x - c[0])**2 + (y - c[1])**2)
    # 全データ点座標から計算した半径の平均値と点ごとの差。これを最小とする
    def fun(c):
        ri = calc_radius(c)
        return ri - ri.mean()
    # 初期推定値
    center_estimate = np.mean(x), np.mean(y)
    # LSQFit
    result = least_squares(fun, center_estimate)
    center = result.x
    radius = calc_radius(center).mean()
    residuals = calc_radius(center) - radius
    radius_error = np.sqrt(np.sum(residuals**2) / len(residuals))
    return center, radius, radius_error
```